ATIVIDADE DE REVISÃO – ALICIA DIAS

Desenvolvimento Web III

1. Fetch (POST)

Esse código cria uma função que coleta o valor digitado no input e monta um objeto. Depois, usa o fetch com método POST para enviar os dados ao backend em formato JSON.
Por fim, confirma no console que o item foi enviado.

```
async function adicionarItem() {
  const item = {
    nome: document.getElementById("nome").value
  };

  await fetch("http://localhost:3000/items", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(item)
  });

  console.log("Item enviado!");
}
```

2. Prisma (ORM)

Define um modelo de tarefa para banco relacional com id automático. Os campos `title` e `done` representam os dados da tarefa, com valor padrão para `done`.
O Prisma usa isso para gerar tabelas e tipos automaticamente.

Mongoose (ODM)

Cria um schema para MongoDB definindo os campos da coleção. O campo `done` já inicia como `false` por padrão. Depois, o modelo é exportado para ser usado no backend.

```
model Task {
  id    Int     @id @default(autoincrement())
  title String
  done  Boolean @default(false)
}

const mongoose = require("mongoose");

const TaskSchema = new mongoose.Schema({
  title: String,
  done: { type: Boolean, default: false }
});

module.exports = mongoose.model("Task", TaskSchema);
```

3. Rota PUT (Express)

Define uma rota que recebe o id da tarefa pela URL. Usa `findByIdAndUpdate` para atualizar todos os dados com base no corpo da requisição.
Retorna a tarefa já atualizada como resposta.

```
app.put("/tasks/:id", async (req, res) => {
  const { id } = req.params;

  const updatedTask = await Task.findByIdAndUpdate(
    id,
    req.body,
    { new: true }
  );

  res.json(updatedTask);
});
```

4. Email único (Mongoose)

Cria dois usuários com o mesmo email e tenta salvar ambos. O primeiro funciona normalmente, mas o segundo gera erro de duplicidade. Isso acontece porque o campo foi definido como `unique`.

```
const user1 = new User({ email: "teste@email.com" });
await user1.save();

const user2 = new User({ email: "teste@email.com" });
await user2.save();
```

5. Quadro comparativo

Mostra a diferença entre Prisma e Mongoose em formato de tabela. Prisma trabalha com bancos SQL e Mongoose com MongoDB. Ambos fazem o mapeamento de dados para objetos no código.

Ferramenta	Tipo	Banco	Exemplo
Prisma	ORM	SQL	PostgreSQL
Mongoose	ODM	NoSQL	MongoDB

6.

Resposta: DELETE
Explicação: É o método HTTP padrão para remover dados permanentemente.

7.

Resposta: Task.find()
Explicação: No Mongoose, .find() retorna todos os documentos de uma coleção.

8.
Resposta: PATCH
Explicação: Usado para atualização parcial (apenas um campo).

9.

Resposta: Fazer a ponte entre aplicação e MongoDB
Explicação: O Mongoose mapeia objetos JS para documentos no banco.

10.
Resposta: DELETE
Explicação: Representa semanticamente a exclusão de dados.

11.

Resposta: GET
Explicação: GET é usado para buscar/retornar dados.

12.
Resposta:

```
senha: { type: String, required: true }
```

Explicação: Define como texto e obrigatório.

13.
Resposta: B
Explicação: Usa @id, autoincrement() e tipo Decimal corretamente.

14.
Resposta: PATCH
Explicação: Alteração de apenas um campo específico.

15.
Resposta: D) shoppingitems
Explicação: Prisma cria coleção em minúsculo e plural.

16.
Resposta: C) npx prisma studio
Explicação: Abre interface visual do banco.

17.
Resposta: C
Explicação: Executa arquivos .ts diretamente sem compilar manualmente.

18.
Resposta: C
Explicação: ? indica campo opcional (pode ser nulo).

19.
Resposta: D) .env
Explicação: Armazena variáveis sensíveis como DATABASE_URL.

20/21.
Resposta: C
Explicação: Tentou inserir email duplicado em campo @unique.

22.
Resposta: C
Explicação: Retorna até 5 usuários ativos.

23.

Resposta: A
Explicação:
migrate dev → SQL com histórico
db push → usado direto (comum em MongoDB)

24.
Resposta: C) dist
Explicação: outDir define a pasta de saída.

25.
Resposta:
Introspection é quando o Prisma lê um banco já existente e gera o schema automaticamente.
Explicação: Usado quando o banco já está pronto.

26 (Categorias e Tarefas)
Resposta:

```
model    Category {
  id     String @id @default(auto()) @map("_id") @db.ObjectId
  name   String
  tasks  Task[]
}

model         Task {
  id          String @id @default(auto()) @map("_id") @db.ObjectId
  title       String
  category    Category @relation(fields: [categoryId], references: [id], onDelete: Cascade)
  categoryId  String @db.ObjectId
}
```

Explicação: onDelete: Cascade remove tarefas junto com a categoria.

26 (Author e Book)
Resposta: B
Explicação: include traz os relacionamentos.

27.
Resposta:

```
if (quantity < 0) {
  throw new Error("Quantidade não pode ser negativa");
}
```

Explicação: Validação antes de salvar.

28.
Resposta:
prisma.$transaction([]) garante que tudo ocorra junto.
Explicação: Se uma operação falhar, tudo é desfeito (rollback).

29.

Resposta:
Prisma gera tipos automaticamente via @prisma/client.
Explicação: Tipos são inferidos com base no schema.

30.
Resposta:
Type-safe evita erros de tipo em tempo de desenvolvimento.
Explicação: Detecta problemas antes de rodar.

31.
Resposta: C
Explicação: Será _id automático do MongoDB (ObjectId).

32.
Resposta: C
Explicação: Retorna o objeto criado com ID.

33.
Resposta:

```
isDone: true
```

Explicação: Filtra itens concluídos.

34.
Resposta: B) @unique
Explicação: Garante valores únicos.

35.
Resposta:
npm install @prisma/client
npx prisma generate
Explicação: Instala e gera o client do Prisma.